枝刈り・CSRによる色補正モデルの圧縮

アジャイルワーク2　実験報告書

25G1017　岩澤大心

記録整理日：2026年11月12日

1　概要

本実験では、紙に印刷した色をセンサで読み取り、元の色に近づける色補正プログラムを扱った。色の補正には、入力と正解の対応を学習したニューラルネットワークを使う。学習済みの補正用データと計算の仕組みを、モデルと呼ぶ。

モデルを大きくすると、保存する数値と計算量が増える。そこで、小さい重みを0にする「枝刈り」と、0以外の値だけを保存する「CSR」を組み合わせた。元の通常方式（Dense）と、枝刈り＋CSRを使った圧縮方式を比べた。

モデルの大きさを40・60・80・100の4種類とし、紙の9色を10回読んだ。各回の同じ読取データを両方式へ入力し、色の誤差を調べた。別にArduino上の計算時間とメモリ使用量も測った。

結果として、モデルの保存容量は60.51〜69.70%、計算時間は28.97〜44.01%減った。1モデル入りプログラム全体のFlashも7.94〜31.26%減った。一方、色の誤差を表すMSEは、10回平均で通常方式が231.68〜319.56、圧縮方式が452.04〜651.93となった。MSEは小さいほどよいため、**容量と速度は改善したが、色を正確に補正する性能を保つ目標は達成しなかった**。静的RAMは全8条件で6,764 byteと同じだった。

2　目的

Arduinoは、使える保存領域と作業用メモリに限りがある小型の計算機である。モデルを大きくしても使い続けるには、数値の保存方法と計算方法を工夫する必要がある。

本実験の目的は、枝刈り＋CSRによって、次の3点がどの程度変わるか確かめることである。

1. **容量**：補正に必要な数値を保存する領域を小さくできるか。
2. **速度**：1色を補正する計算時間を短くできるか。
3. **正確さ**：容量と速度を改善しても、正解の色との誤差を小さく保てるか。

重みを0にするだけでは、配列に0を保存する場所が残る。CSRも組み合わせることで、保存する値と計算する接続の両方を減らす。同じ大きさのモデルへ同じ入力を与え、圧縮による変化を比べた。

正確さの比較条件は、通常方式に対するMSEの増加が5%以下であることとした。加えて、利用者の目標であるMSE300以下も別に判定した。当初計画は40・50・60次元だったが、実施条件は40・60・80・100であり、50は未実施である。

3　実験の全体像と用語

3.1　同じ読取データを2つの方式へ入れる

読み取った色を赤・緑・青の3つの数値（RGB）で表し、モデルへ入れると、補正後のRGBが出る。学習済みモデルを使って出力を求める計算を推論と呼ぶ。

図1のように、1回読み取ったデータを通常方式と圧縮方式の両方へ渡した。方式ごとに紙を読み直すと、紙の位置や読取時刻の違いも結果に入る。同じ入力を共有することで、両方式の計算結果を比べられる。

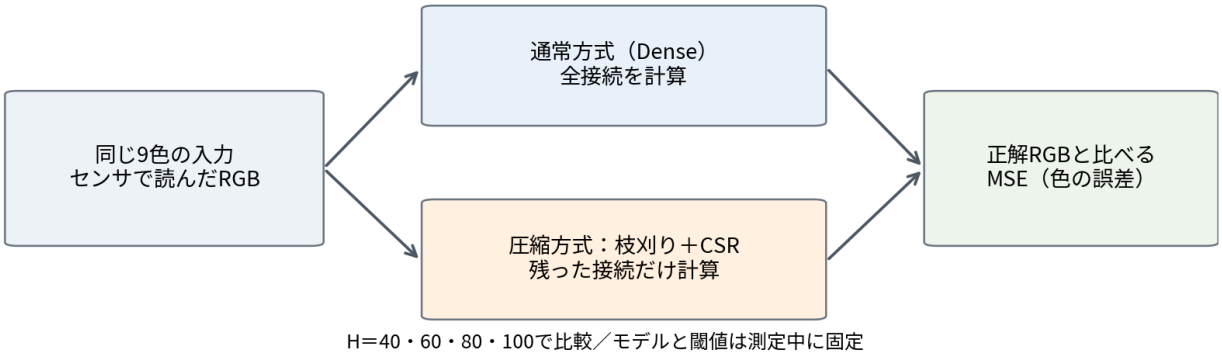


図1　同じ入力で2方式を比較する流れ。

40・60・80・100の4種類に、通常方式と圧縮方式を組み合わせたため、比較条件は合計8個である。1回の紙の読取から、8条件分の結果が得られる。

3.2　このレポートで使う言葉

用語	意味
重み・バイアス	重みは入力に掛ける係数、バイアスは足し合わせる調整値
次元H	2つの隠れ層それぞれの計算単位の数。H40は各層40個であり、紙の色数ではない
Dense（通常方式）	0も含めて全重みを配列に保存し、全接続を順に計算する方式
枝刈り	絶対値が設定値より小さい重みを0にする処理
CSR	0以外の重みと、その位置だけをまとめて保存する方法
閾値（しきい値）	重みを0にするかどうかを決める境目の値
MSE	正解とのRGBの差を二乗して平均した値。小さいほどよく、0なら全て一致
FP32	1つの実数を32 bit＝4 byteで保存する形式。両方式で共通
Flash・RAM	Flashはプログラム等の保存領域、RAMは動作中の値を置く作業領域

byte（バイト）は容量の単位で、図の1 KiBは1,024 byteである。時間の1 µsは100万分の1秒、1 msは1,000 µsである。

3.3　データの用途を分ける

学習はモデルの重みを決める作業、閾値選択はどの重みを0にするかを調整する作業である。その後に、決めたモデルを変えずに紙を新しく読んだ。

用途	使ったデータ
学習	過去に実測した135行。3×3チャート分111行、6×4チャート分24行
閾値選択	別の8×8チャートを2回読んだ128入力。学習内9入力でMSE300以下も確認
正確さの測定	学習にも使った3×3の紙を新しく10回読取。各回9色で入力は計90組
速度の測定	以前に保存した実測9入力を、全モデルへ繰り返し入力
単独メモリ比較	モデル1個ずつを入れた8条件のプログラムをコンパイル

新規10回の入力は学習や閾値選択へ加えていない。ただし紙の色自体は学習時と同じである。そのため今回は同じ紙を読み直した場合の比較であり、初めて見る色の紙でも正確に補正できるかという最終確認は行っていない。

4　仕組みと計算方法

4.1　ネットワークと学習

ネットワークは「RGBの3入力→H個の計算→H個の計算→RGBの3出力」という構造である。途中の2段を隠れ層と呼ぶ。Hを大きくすると計算の組合せが増えるが、保存する重みと計算量も増える。

入力を x、途中の結果を h_1, h_2、出力を y とする。各段の重みを行列 W、バイアスを b で表すと、計算は次のようになる。

$$
\begin{aligned}
h_1 &= \mathrm{ReLU}(W_1 x + b_1),\\
h_2 &= \mathrm{ReLU}(W_2 h_1 + b_2),\\
y &= W_3 h_2 + b_3.
\end{aligned}
$$

行っているのは、「前の段の値に重みを掛けて足し、バイアスを加える」という処理である。ReLUは、結果が負なら0、0以上ならその値を通す処理を表す。最後の段にはReLUを使わない。

学習では、出力を正解へ近づけるように重みを調整する。学習で小さくしたい誤差の指標を損失と呼ぶ。本実験では、Smooth L1損失に、重みを小さくするL1正則化[1]を加えた。L1正則化は重みの絶対値の合計へ罰則を加える方法で、この後の枝刈りで小さな重みを0にする準備となる。学習だけで必ず0になるとは限らない。

実際の学習式を次に示す。$M = 135$ は学習行数、a_i は各行をどれだけ重視するかという正の値、$\hat{y}$ は予測値、t は正解値、c はRGBのいずれかである。学習時の予測と正解は0〜1の尺度を使う。a_i は学習行の重要度で、ネットワーク内の重み W とは別の値である。

$$
\mathcal{L} = \frac{1}{\sum_{i=1}^{M} a_i} \sum_{i=1}^{M} \frac{a_i}{3} \sum_{c \in \{R,G,B\}} \ell_\beta(\hat{y}_{i,c} - t_{i,c}) + 10^{-6} \sum_{\ell=1}^{3} \sum_{j,k} |(W_\ell)_{jk}|,
$$

$$
\ell_\beta(e) = \begin{cases} e^2/(2\beta) & (|e| < \beta), \\ |e| - \beta/2 & (|e| \geq \beta), \end{cases} \qquad \beta = 0.05
$$

ここでeは予測と正解の差、beta（β）は二乗から直線へ切り替わる境目で、0.05に設定した。ℓは層番号、jとkは重みの表の中の位置を表す。

式の前半は「全学習行のRGB誤差を、行ごとの重要度に従って平均する」部分である。Smooth L1[5]は小さな誤差では二乗、大きな誤差では直線的に増えるため、一部の大きな誤差が学習を強く支配することを抑える。

後半は「全3層の重みの絶対値を足し、10^{-6} 倍して加える」部分である。$\sum$は指定範囲の値を全て足す記号で、層とその中の各重みを順番に足している。この罰則をバイアスには加えない。

この式は学習時の調整に使うものであり、結果の正確さは整数RGBのMSEで評価する。

4.2 枝刈りとCSR

枝刈り[2]では、重みの絶対値が閾値 τ より小さければ0にする。説明用に閾値を0.1とすると、0.03の重みは0になり、0.2は残る。負の重みでも絶対値で判断し、バイアスは変えない。

その後、残った重みをCSR[3]へ変換する。保存するのは、0以外の重み、その列番号、各行の開始位置である。ここでの行と列は、重みを表に並べたときの位置を指す。位置情報があれば、0を省いても残した重みをどの入力に掛けるか分かる。具体例は実装の節で示す。

3層で残った重みの個数をK、0にした重みの割合をSとすると、

$$B_{\mathrm{CSR}} = 5K + 12H + 24 \quad [\mathrm{byte}], \qquad S = 1 - \frac{K}{H^2 + 6H}$$

となる。容量の式の内訳は次節で計算する。Sは表3の「疎性」に当たり、大きいほど0にした接続が多い。

4.3 メモリ容量の手計算

容量は「保存する数値の個数×1個当たりのbyte数」で求める。最初の段は3入力からH個へつなぐため重みが3H個、次はH個からH個でH^2個、最後はH個から3出力で3H個になる。

したがって全重み数は $N_w = 3H + H^2 + 3H$、全バイアス数は $N_b = H + H + 3$ である。表の形で書くと、重みの行列は順に $H \times 3$、$H \times H$、$3 \times H$ となる。Denseは1個を4 byteで保存するので、容量は $4(N_w + N_b) = 4(H^2 + 8H + 3)$ byteである。

CSRは残った重みK個だけを保存するため、重みが4K byte、その位置を示す列番号がK byte必要になる。列番号は1 byteの符号なし整数（uint8）を使った。

行の開始位置は2 byteの符号なし整数（uint16）を使う。各層の行数に終端1個を足すため、個数は $(H+1)+(H+1)+(3+1) = 2H+6$ 個、容量は $2(2H+6)$ byteである。バイアスは全て残すので $4(2H+3)$ byteになる。4種類の容量を足すと、

$$\begin{aligned} B_{\mathrm{CSR}} &= 4K + K + 2(2H+6) + 4(2H+3) \\ &= 5K + 12H + 24 \quad [\mathrm{byte}]. \end{aligned}$$

例えばH60では、元の重みは3,960個、バイアスは123個である。枝刈り後に残った重みはK=1,141個だった。これらを代入すると、

$$\begin{aligned} B_{\mathrm{Dense}} &= 4(60^2 + 8 \cdot 60 + 3) = 16,332, \\ B_{\mathrm{CSR}} &= 4,564 + 1,141 + 252 + 492 = 6,449, \\ R_B &= 100(1 - 6,449/16,332) = 60.51\%. \end{aligned}$$

各Hについて同じ計算をした内訳が次表である。列の「非ゼロ重み」は、個数ではなく重みを保存する容量を示す。全列の単位はbyteで、合計がCSRのモデル配列容量になる。

H	非ゼロ重み	列番号	行位置	バイアス	合計byte
40	1,880	470	172	332	2,854
60	4,564	1,141	252	492	6,449
80	6,596	1,649	332	652	9,229
100	9,496	2,374	412	812	13,094

重み1個を残すと値4 byteと列番号1 byteが必要なので、CSRに変換するだけで必ず小さくなるわけではない。上式より、配列容量が小さくなる条件は $5K < 4H^2 + 20H - 12$ である。採用した4条件はこの条件を満たす。

ここで数えたのは配列の数値だけである。データをまとめる管理情報（構造体）、配置をそろえる余白（アラインメント）、Arduinoの基本処理などを含むFlash全体は、IDE表示で別に評価した。モデル配列が減った割合を、そのまま静的RAMが減った割合とはしない。

4.4　評価指標

色の正確さはMSEで調べる。 モデルの出力を0〜1の範囲に収め、255倍して四捨五入し、0〜255の整数RGBに変換する。0未満を0へ、1を超える値を1へ収める処理をクリップと呼ぶ。

9色にはRGBがそれぞれ3成分あるため、比較する数値は9×3＝27個である。補正後の値を y、正解を t とすると、

$$\mathrm{MSE} = \frac{1}{27} \sum_{i=1}^{9} \sum_{c \in \{R,G,B\}} (y_{i,c} - t_{i,c})^2$$

で求める。つまり、27個それぞれの差を二乗して足し、27で割る。計算例として、27個全ての差が10ならMSEは100となる。0なら全て一致しており、小さいほど正解に近い。

10回の結果は、平均とばらつきを合わせて見る。 1回ごとのMSEを m_r とし、平均と標本標準偏差（SD）を次で計算する。

$$\overline{m} = \frac{1}{10} \sum_{r=1}^{10} m_r, \qquad s_m = \sqrt{\frac{1}{9} \sum_{r=1}^{10} (m_r - \overline{m})^2}.$$

平均は全10回をまとめた誤差の大きさ、SDは各回の結果がどれだけ散らばったかを表す。SDが大きいほど読取ごとの違いが大きい。標本SDでは、平均との差の二乗を足して10−1＝9で割る。

同じ回で「圧縮方式のMSE−通常方式のMSE」を求めた差を d_r とした。正の値なら、圧縮後に誤差が増えたことになる。MSE増加率は各回の割合の平均ではなく、$100(\overline{m}_{\mathrm{CSR}}/\overline{m}_{\mathrm{Dense}} - 1)$ で求める。

容量Bと計算時間Tの削減率は、それぞれ $100(1 - B_{\mathrm{CSR}}/B_{\mathrm{Dense}})$ と $100(1 - T_{\mathrm{CSR}}/T_{\mathrm{Dense}})$ である。例えば容量が100から60へ減れば、削減率は40%となる。

5　実験方法

自分で変えて比較する条件（独立変数）は、次元Hと通常／圧縮の方式である。その結果として調べる値（従属変数）は、MSE・容量・Flash・静的RAM・計算時間である。同じHでは、元のモデル、入力、RGBの丸め、ボード、計時条件をそろえた。H間では追加学習を始めた時点の重みも違うため、H以外の影響が全くない比較とはいえない。

5.1 装置と実行環境

項目	実施条件
ボード	Arduino UNO R4 WiFi
主マイコン・クロック	Renesas RA4M1、48 MHz
公称Flash／SRAM	262,144／32,768 byte
センサ	A0へ接続した光センサ。型番は未確認
LED	R:D8、G:D6、B:D7。analogWrite 255／0
読取設定	各チャンネル10サンプル平均、サンプル間2 ms、点灯後10 ms待機
操作	シリアル9600 bps。外付けD2/D3ボタンは使わない
利用者のPC	Mac。Pythonの利用環境はJupyter Notebook
最終学習・集計	クラウドPython 3.12、PyTorch 2.8.0、NumPy 2.3.5
実機コンパイラ	7.2.1 20170904、ARM/embedded-7-branch revision 255204
Arduino IDE・コアの版	版番号は未記録。実行ログとコンパイラ文字列を保存

紙は手で支えて読み取った。照明を同じ状態に保つ手順としたが、照度・距離・角度の定量記録はない。固定具を用いた測定ではなく、この条件は再現性の制約となる。

5.2 学習データと正解画像

学習データの1行は、実際のセンサ値から作った入力と、その色の正解RGBを1組にしたものである。3×3の紙を読んだ111行と、6×4の紙を読んだ24行を合わせ、135行全てを残した。過去の入力値は変えず、配布Notebookの見本入力を新しい実測として加えてはいない。

PPMはRGBの数値を並べて画像を表すファイル形式である。正解画像の取り違えが判明したため、最終V10では3×3の111行を先生指定の元の `result.ppm` のRGBへ修正した。正解PPM自体は変更していない。下段中央の灰色は(127,127,127)である。6×4の24行は元のNotebookの教師値を保持しており、別の先生指定PPMとの照合はできていない。以前のCMYK基準のMSEを、今回の基準での改善値へ混ぜない。

各Hでは、前の版V9で保存した同じHの重みから追加学習し、最終版をV10とした。学習データを繰り返し使う回数は40,000 epoch、乱数の開始条件はseed42に固定した。更新方法はAdam、更新の大きさを決める学習率は0.0005から0.000005へcosine形で変化させた。色・校正グループや古い測定の整合度に基づいて、各行の重要度を調整した。全行の重要度は正で、学習から削除した行はない。

入力変動への対応として、学習時にだけ入力例を変化させた。これはデータ拡張と呼ばれ、新しい紙の測定ではない。3×3入力の50%にRGB別の0.75〜1.30倍の掛け算（gain）と、範囲を制限した足し算（offset）を加えた。灰・白には30%の確率で共通の0.5〜1.15倍のgainを加えた。offset幅は学習入力同士の距離から決め、設定と各行の重要度はコード・CSVへ保存した。

学習へ追加していない保存済み実測9入力を用いてPC・実機再生の照合も行ったが、これは以降の新規10回とは別の記録である。新規10回の入力は学習・閾値選択へ一切加えていない。

5.3　実装アルゴリズムとソースコード

実装は、PCでモデルを準備する部分と、Arduinoで色を補正する部分に分けた。PCでは学習した重みから閾値を選び、小さな重みを0にしてCSRへ変換する。用意した数値は、コードとは別のヘッダファイル（.h）に保存した。Arduinoではこのファイルを読み、同じ入力から通常方式と圧縮方式の結果を出す。保存する数値の形式は、両方式ともFP32でそろえた。

CSRで保存する3つの配列。 配列は、数値を番号順に並べたものである。重みの表では、1行が1つの出力先、1列が1つの入力元に対応する。上の行から順に調べ、0以外の重みを `values`、その列番号を `columns` へ追加する。`rows` には、それぞれの行がvaluesの何番から始まるかを記録する。プログラムの番号は0から数える。

説明用に、実測モデルとは別の小さな行列を考える。

$$W = \begin{pmatrix} 0 & 0.5 & 0 \\ -0.2 & 0 & 0.1 \end{pmatrix}.$$

$$\begin{aligned} \text{values} &= [0.5, -0.2, 0.1], \\ \text{columns} &= [1, 0, 2], \quad \text{rows} = [0, 1, 3]. \end{aligned}$$

この表には6個の重みがあるが、0以外は0.5、−0.2、0.1の3個だけなので、それだけをvaluesに保存する。columnsは元の列番号を示す。rowsの[0,1,3]は、「1行目は0番から1番の手前まで」「2行目は1番から3番の手前まで」を意味する。最後の3は、全要素の終わりを示す値である。

例えば入力が (x_0, x_1, x_2) なら、1行目は $0.5x_1$、2行目は $-0.2x_0 + 0.1x_2$ と計算できる。0を省いても、残った値の使い場所が分かることが重要である。実装では各行の列番号を小さい順に保存し、足す順序を元の行列とそろえた。

ArduinoのCSR推論。 次は実際の `teacher_compare_v10.ino` にある関数である。`comparison_models`という名前でまとめた処理に含まれ、`layer.col`はcolumns、`layer.row`はrowsを参照する。

```
void sparseLayer(const comparison_models::SparseLayer &layer,const float *input,float *output,bool
↪  relu) {
  for(int r=0;r<layer.rows;r++) {
    float value=layer.bias[r];
    for(uint16_t k=layer.row[r];k<layer.row[r+1];k++) value+=layer.values[k]*input[layer.col[k]];
    output[r]=(relu && value<0)?0:value;
  }
}
```

コードのrは行番号、kは保存した重みの番号である。`value=layer.bias[r]`でバイアスを用意し、その行に残った重みと対応する入力を掛けて足す。`layer.col[k]`が入力の使い場所を示す。隠れ層では最後にReLUを通し、出力層では通さない。

`trained_models::predict()`は全接続を調べる通常方式の関数である。`comparison_models::predict()`は、条件番号に応じて通常方式か3層のCSR計算を呼び出す。途中の結果は2本の配列へ一時保存する。

入力・出力と検査。 `normalizeInput()` は黒白校正値による正規化と0〜1へのクリップを行う。`comparison_models::inferAll()` は1組の入力から全8条件を計算する。出力を整数RGBへ変換してPPMを出力し、`result.ppm` と同じ色順でMSEを求める。出力色を色番号で置き換える処理は行わない。

`validIndices()`は、行の開始位置が順に並んでいるか、終端が保存個数と合っているか、列番号が範囲内かを調べる。`selfCheckAll()`は、同じ入力から想定した数値が出るかを確認する。

検査は2種類ある。**CSRへの保存変換が正しいか**は、枝刈りで0にした後の重みを通常の配列で計算した結果と比べる。**枝刈りで色の誤差が増えたか**は、枝刈り前の通常方式と比べる。変換が正しくても、枝刈りによる誤差は増える場合がある。

主なファイルの役割は次のとおりである。数値ヘッダはコードと別ファイルに保存した。

ファイル	主な役割
train.py	各Hの追加学習、重みと学習条件の保存
select_compression.py	別の検証128入力による閾値候補の比較
finalize_compression.py	学習内9入力の追加条件を確認し、閾値を固定
build_program.py	固定したDense重み・CSR配列と参照値をヘッダへ出力
teacher_compare_v10.ino	保存入力の実機再生、MSE照合、速度測定
teacher_fresh_v10.ino	黒白校正、新規9色の読取、全8条件の出力
memory_probe_v10.ino	指定した1条件だけを含むFlash・静的RAMビルド
record_fresh_capture.py	受領ログの入力・出力・MSEの検算と10回の集計
record_single_model_memory.py	条件名付きIDE表示の保存、容量と削減率の集計

5.4 閾値選択

閾値が大きいほど多くの重みを0にできるが、誤差も増えるおそれがある。そこで重みの絶対値を調べ、隣り合う異なる値の中間を候補にした。候補数は5個などに固定していない。

各候補を別の8×8チャートの128入力で試し、「MSE増加5%以下」と「容量がDenseより小さい」を満たすものを残した。さらに、学習内のV8という版で読んだ9入力でもMSE300以下になる候補から、最小容量のものを選んだ。調整を終えてから新規10回へ進み、10回の結果を見て閾値を選び直してはいない。

下表の検証は閾値を調整した128入力、最後の列は追加条件に使った学習内9入力の結果である。後の新規10回とは入力が異なる。

検証2回目では異なる色が同一の0入力へ潰れる問題が確認されており、閾値の選択は暫定とした。検証128入力で条件を満たすことは、新規10回で精度を維持できる保証にはならない。

H	検証Dense MSE	検証CSR MSE	変化率	学習内V8の9入力CSR MSE
40	5012.20	4715.84	-5.91%	277.52
60	5226.81	5292.59	+1.26%	260.19
80	5135.44	4553.44	-11.33%	259.26
100	5237.22	4632.38	-11.55%	290.37

5.5 紙の新規10回読取

新規読取版 TEACHER_FRESH_V10 を使用した。各回で黒を当ててb、白を当ててwを送り、センサの基準を合わせた。この基準合わせを校正という。

読取値v、黒の値b、白の値wから、各色成分の入力を $(v-b)/(w-b)$ で求めた。黒の基準が0、白の基準が1になるようにするためである。値の範囲をそろえる処理を正規化と呼び、0〜1から外れた値はクリップして入力した。

その後、赤・緑・青・黄・水色・紫・黒・灰・白の順に紙を当て直し、各色でrを1回送信した。1色の同じ新規入力を全8モデルに与え、9色終了後に8個のMSEと8枚のPPMを出力した。ログ保存後、nで次の校正・読取へ進み、CAPTURE=1〜10を実施した。1回は9色のチャート読取で、各色の内部10サンプル平均を反復回数として数えない。

全10回の読取直後の値（RAW）・黒白校正値・正規化入力・モデルの識別値（ハッシュ）・閾値・出力PPMを保存した。重み・閾値は途中で変更していない。第2回はMSEが高かったため操作ミスの有無を確認したが、「気づいた操作ミスはない」との回答であり、この回を含む全10回を集計した。除外した回はない。

5.6　速度と容量の測定

速度は別の TEACHER_RESULT_COMPARE_V10 で、固定した保存実測9入力を巡回して実機で計時した。各モデルを10回ウォームアップし、100推論を1組として5組計時した。Dense先行とCSR先行を交互に切り替えた。推論と共通のチェックサム加算を含み、センサ読取・シリアル出力・MSE計算を除いた。新規10回の各回で推論時間を計ったわけではない。

micros() [4]は動作開始からの時間をμsで読む関数である。推論ループの前後の差を100で割り、1色当たりの計算時間を求めた。実際の計時関数は次のとおりである。比較対象で共通の入力選択とチェックサム加算も区間内に含むため、この測定値を推論関数単体だけの時間と読み替えない。

```
uint32_t timeModel(int model,int group,double &checksum) {
  float output[3];checksum=0;
  uint32_t begin=static_cast<uint32_t>(micros());
  for(int call=0;call<100;call++) {
    int input=(group*100+call)%comparison_models::BENCHMARK_INPUT_COUNT;
    comparison_models::predict(model,comparison_models::BENCHMARK_INPUTS[input],output);
    checksum+=output[0]+output[1]+output[2];
  }
  return static_cast<uint32_t>(micros())-begin;
}
```

コードのbeginは開始時刻、predict()は色の補正、最後のmicros()-beginは100推論の経過時間である。checksumは出力の合計を取り、計算結果が使われるようにするための値である。

容量は、モデルの数値配列だけの容量、全8条件入りプログラム全体の容量、1条件ずつ入れたプログラムの容量に分けて調べた。配列容量には、重み・バイアス・CSRの列番号・行の開始位置を含めた。

その後、単独モデル用 memory_probe_v10 を用い、H=40・60・80・100とDense／CSRの8条件を1条件ずつコンパイルした。重み・閾値は最終V10のままとし、各条件名に対応するArduino IDEの「スケッチ」「グローバル変数」の表示を記録した。共通の初期化・推論呼出し・シリアル出力を含む1モデル入りプログラム全体の比較であり、センサ読取とMSE計算を含まない。この追加操作で紙の測定回数は増えていない。単独ビルドのIDE・ボードコア版とバイナリは未記録である。

5.7　計画から変更した実装とその理由

学習例を増やし、学習時の誤差の扱いを変えた理由。 同じ色でも校正や読取状態によって入力が変わり、最初の33組だけでは、その変化に十分対応できなかった。そこで追加読取を含む135行を残し、学習時に掛け算・足し算で入力を変化させた。

また、二乗誤差では、大きく外れた入力が学習の更新へ強く影響する。そこで、大きな誤差では増え方が直線になるSmooth L1[5]と、行ごとの正の重要度を使った。大きな誤差の行を削除する代わりに、その影響を調整するためである。結果の評価は引き続きMSEで行い、学習全体や新規読取に大きな誤差が残ったことも報告した。

正解画像と閾値の条件をそろえた理由。 Notebookの表示画像と先生指定の result.ppm は、RGBが異なっていた。同じ予測でも比べる正解が違えばMSEが変わるため、3×3の教師値と評価を先生指定PPMへ統一した。

閾値には、検証MSEの増加5%以下に加えて、学習内9入力でMSE300以下も確認する条件を付けた。利用者の誤差目標を学習内でも確かめるためである。ただし、それだけで別の読取も正確になる保証はなく、新規10回の圧縮方式の平均MSE300以下は達成できなかった。

次元と保存場所。 40・60に加えて80・100を比較し、Hを大きくしたときの容量・速度・誤差を調べた。50は未実施であり、80・100の測定で50の比較まで終わったとは扱わない。

授業例の「60次元で可、70次元で不可」は、数値の置き場所や作り方が違えば変わり得る。今回は変更しない定数配列としてモデルを用意し、H100の通常方式で数値配列43,212 byteに対し、静的RAM表示は6,764 byteだった。モデルの数値配列の容量と、RAMの表示を直接同じものとはみなせない。実際の搭載限界と、数値の配置を詳しく示すリンカマップは未測定・未保存である。

評価範囲。 独立した最終チャートを使う計画だったが、最終測定日に追加チャート・印刷を利用できなかったため、同じ学習チャートの新規10回読取による対応比較を行った。これは同一入力に対するDense／CSRの差と読取のばらつきを調べるためには使えるが、未知の色への汎化性能の検証を代替しない。紙の読取を繰り返した回数と、保存入力を実機で再生した回数も分けて記録した。

6　結果

6.1　新規10回のMSE

表1は全10回の平均とばらつきである。DENSEは通常方式、CSRは圧縮方式を表す。「300以下の回数」は目標を満たした回数である。紙の読取は10回で、そこから8条件の結果を出した80個のMSEを、80回の紙の測定とは数えない。

表1　新規10回の精度。

H	方式	平均MSE	標本SD	最小	最大	300以下の回数
40	DENSE	319.56	903.46	5.30	2889.15	9/10
40	CSR	452.04	923.64	112.70	3078.33	9/10
60	DENSE	238.34	738.24	1.22	2339.37	9/10
60	CSR	493.51	814.81	177.19	2811.56	9/10
80	DENSE	231.68	646.48	1.26	2069.26	9/10
80	CSR	465.73	683.36	202.44	2407.85	8/10
100	DENSE	260.43	643.69	2.44	2063.63	8/10
100	CSR	651.93	694.08	318.85	2576.07	0/10

10回平均でMSE300以下だったのはDenseのH60・80・100であった。CSRは全Hで平均300を超え、H100のCSRは各回でも一度も300以下にならなかった。最終第10回だけの低い値を、10回平均へ置き換えてはいない。

表2の差は「圧縮方式のMSE−通常方式のMSE」で、正なら圧縮後に誤差が増えたことを表す。

表2　同じ回におけるCSR−Denseの差。

H	平均差	差の標本SD	平均MSEの相対増加
40	132.48	25.44	+41.46%
60	255.17	79.34	+107.06%
80	234.05	45.73	+101.02%
100	391.50	68.45	+150.33%

全10回・全HでCSRのMSEはDenseより大きかった。平均MSEの増加率は41.46〜150.33%で、比較条件の5%以内には収まらなかった。

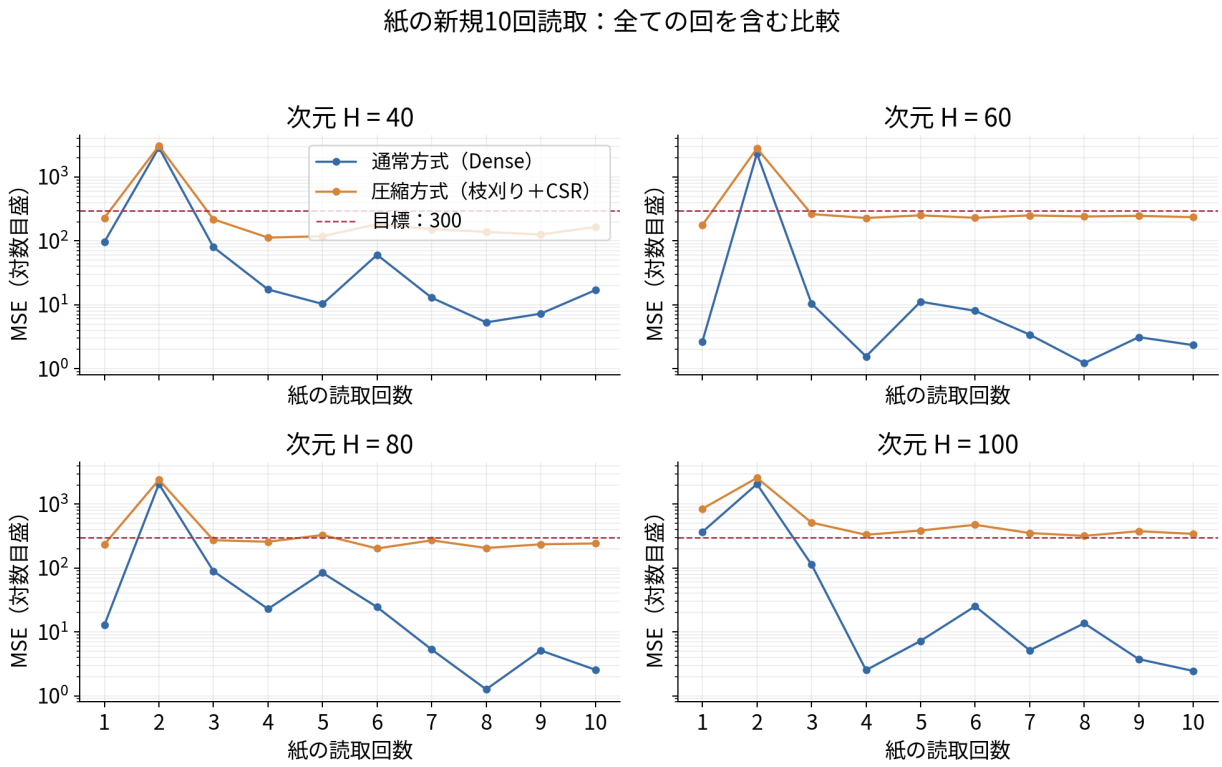


図2　各回のMSE。

下にある点ほど誤差が小さく、破線は目標300である。縦軸は大きく変わる値を比べやすい対数目盛を使い、同じ間隔が同じ倍率を表す。第2回を含む全点を表示した。

6.2　モデル配列容量

表3のNNZは残した0以外の重みの個数、疎性は元の重みを0にした割合である。例えば疎性が約74%なら、元の接続の約74%を0にしたことを表す。位置情報の保存分も含めて比べた。

表3　採用した枝刈り閾値と配列容量。

H	閾値	NNZ	疎性	Dense byte	CSR byte	削減率
40	0.06084981	470	74.46%	7,692	2,854	62.90%
60	0.05434179	1141	71.19%	16,332	6,449	60.51%
80	0.04554720	1649	76.03%	28,172	9,229	67.24%
100	0.05893202	2374	77.60%	43,212	13,094	69.70%

CSR索引を含めても、全Hで配列容量が小さくなった。最大削減率はH100の69.70%であり、H40のCSRは全条件中で最小の2,854 byteであった。この削減率はモデル配列の比較値であり、ファームウェア全体のFlashやRAMの削減率ではない。

6.3　実機推論時間

表4　固定入力での推論時間。単位はμs/推論、±は5組の平均時間の標本SD。

H	Dense 平均±SD	CSR 平均±SD	時間短縮率
40	465.262 ± 0.0179	305.834 ± 0.0913	34.27%
60	945.648 ± 0.0217	671.674 ± 0.0799	28.97%

H	Dense 平均±SD	CSR 平均±SD	時間短縮率
80	1593.368 ± 0.0192	956.212 ± 0.2984	39.99%
100	2408.160 ± 0.0158	1348.392 ± 0.2527	44.01%

全HでCSRの計算が速くなった。例えばH60は945.648→671.674 µs、秒に直すと約0.000946→0.000672秒であり、28.97%短縮した。全Hでの短縮率は28.97〜44.01%だった。表4のSDは各組100推論の平均を5個比べた値で、表1の紙の10回反復によるMSEのSDとは異なる。

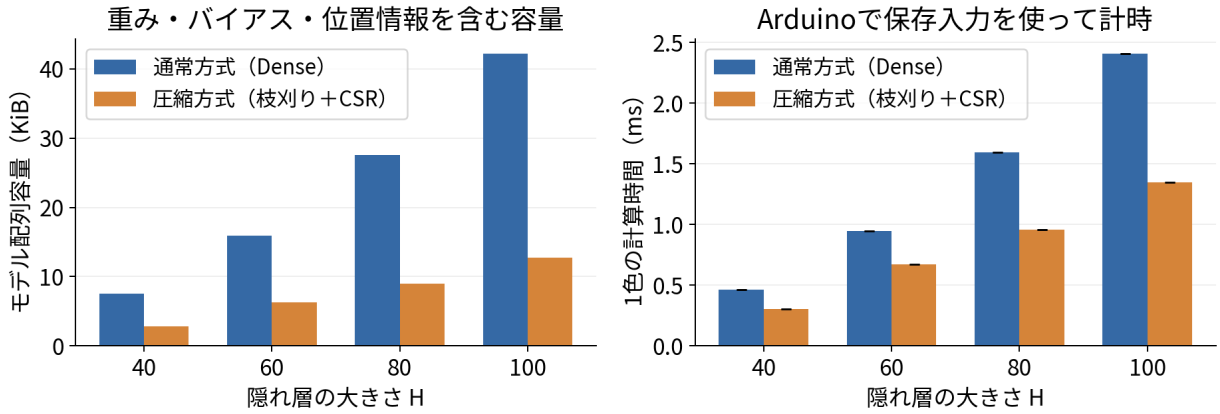


図3　モデル配列容量と推論時間。

左は低い棒ほど保存容量が小さく、右は低い棒ほど計算が速い。時間の棒に付けた短い線は、5組の平均時間のばらつき（SD）を表す。計時には新規10回とは別の固定9入力を使った。

6.4　Flash・RAMと動作確認

Flashは保存領域、RAMは動作中の作業領域である。IDEの「スケッチが…byte」はプログラム全体のFlash、「グローバル変数が…byte」は静的RAMの使用量を示す。静的RAMは、プログラムの動作中に継続して置く変数などの領域である。

表5　8条件入りプログラム全体のArduino IDE使用量。

プログラム	Flash byte	静的RAM byte	静的割当後の残りRAM byte
保存入力再生・速度版	187,888	6,740	26,028
新規読取版・全8条件	191,320	7,324	25,444

公称容量はFlash262,144 byte、SRAM32,768 byteである。表5は全8条件を含むプログラムの使用量である。

表6　各モデル単独ビルドの使用量。単位はbyte。静的RAM列はDense・CSR共通。

H	Dense Flash	CSR Flash	Flash削減量	削減率	両方式の静的RAM
40	60,852	56,020	4,832	7.94%	6,764
60	69,492	59,612	9,880	14.22%	6,764
80	81,348	62,396	18,952	23.30%	6,764
100	96,388	66,260	30,128	31.26%	6,764

2026年11月12日に受領した条件名付きIDE表示に基づく。全HでFlashが減少し、H100では30,128 byte（31.26%）削減した。一方、静的RAMは全8条件で6,764 byte（IDE表示20%）のままで、削減量は0 byteだった。静的割当後に残るRAMは全条件で26,004 byteである。

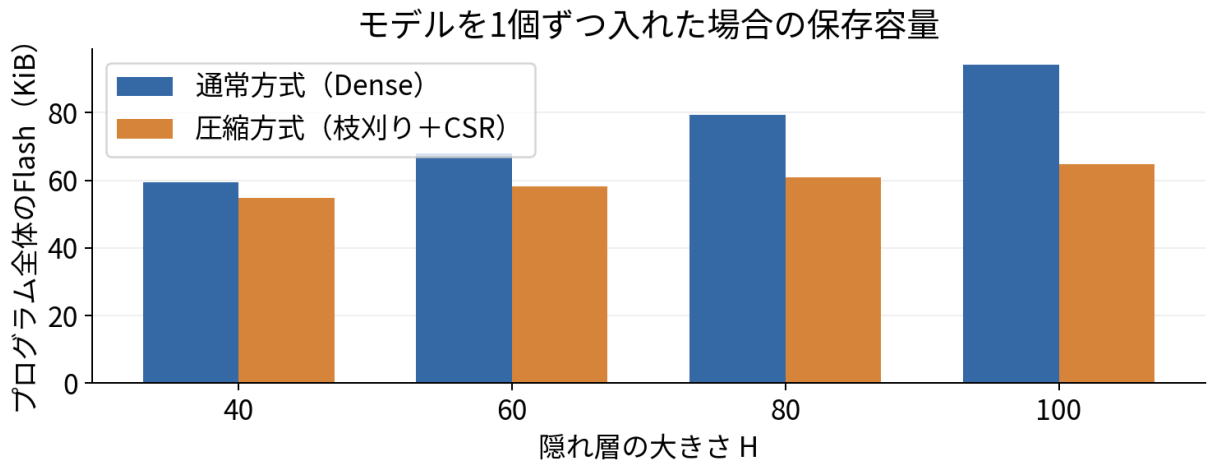


図4　単独モデル入りプログラムのFlash使用量。

Flash全体はArduinoコアや共通処理を含むため、表3の数値配列だけの削減率とは異なる。モデル配列はconstであり、関数内の中間配列は静的RAMに含まれないため、モデル配列の削減から静的RAMの削減を直接導くことはできない。

静的RAMを引いた残りは32,768−6,764＝26,004 byteである。IDEの「ローカル変数で使うことができる」はこの残り容量で、実際に使った量ではない。関数内で一時的に使う作業領域をスタックと呼び、その最大使用量は未測定である。1モデルの隠れ層用配列は固定長float[100]を2本、宣言上800 byteとしたが、これだけで実際の最大スタック使用量を表すことはできない。今回の追加ログはコンパイル時の使用量表示であり、単独版の実行確認・新しい速度測定を行った記録ではない。

全モデルの自己検査とCSR索引検査は保存入力版の実機ログでPASSした。新規10回の全80PPM、計2,160 RGB成分が固定V10のPC再計算と完全一致した。80個のMSEを先生の元の計算HTMLでも照合し、表示桁の範囲で一致した。

7　考察

7.1　容量・速度と正確さを両立できたか

0以外の重みだけを保存・計算したため、モデルは小さくなり、計算も速くなった。一方、0にした重みの影響が出力へ伝わり、今回の閾値では色の誤差が増えた。「小さい重みを削れば正確さへの影響も十分小さい」という見込みは、今回の結果では確認できなかった。

検証128入力ではMSEの増加5%以下という選択条件を満たした一方、新規10回では満たさなかった。異なる色が同じ0入力になる問題や、調整に使った読取と新しい読取の状態の違いなどが考えられる。どの要因が支配的かは本実験だけでは特定できない。

7.2　メモリ・速度・推論性能の定量的な対応

H60で3つの指標を合わせて考える。まず容量は、モデル配列が16,332→6,449 byteで60.51%減った。プログラム全体のFlashは69,492→59,612 byteで14.22%減った。Arduinoの共通処理などが残るため、モデルだけの削減率とプログラム全体の削減率は違う。静的RAMは6,764→6,764 byteで0%削減だった。

次に速度を考える。重みと入力を掛ける回数は、通常方式で $N_w = H^2 + 6H$ 回、圧縮方式でK回になる。H60では3,960→1,141回で71.19%減ったが、計算時間の短縮は945.648→671.674 µsで28.97%だった。

掛け算の回数ほど時間が減らないのは、位置情報を読む処理や繰り返し処理なども必要だからである。全Hでも掛け算の回数は71.19〜77.60%減ったのに対し、時間は28.97〜44.01%の短縮だった。各処理の時間を別々に測ってはいないため、どの処理に最も時間がかかったかは断定しない。

一方、H60の新規10回平均MSEは238.34→493.51で、255.17増加した。差の標本SDは79.34であり、同じ回の対応差は10回とも正だった。H40でもFlashは7.94%、時間は34.27%減ったが、平均MSEは319.56→452.04で41.46%増えた。したがって、容量・速度の改善と精度維持を同時に満たした条件は今回の4組にはなかった。

7.3 第2回とばらつき

第2回の全モデルでMSEが大きく、平均と標本SDへ強く影響した。例えばH60のDenseは最小1.22に対して最大2,339.37、平均238.34、標本SD738.24である。このようなデータでは、最良の1回だけで性能を説明できない。

第2回では青・水色などのRAW・正規化入力も他の回から変わっていた。実機出力は同じ入力からのPC出力と一致しており、MSE表示の計算だけを原因とする差ではない。校正・照射・紙の位置・センサ応答などは候補となるが、照度や距離を計測しておらず、原因を断定しない。操作ミスの申告がないため、第2回を外れ値として削除しなかった。

7.4 次元数と適用範囲

今回の平均値ではDenseのH80が最小MSEだったが、Hを増やすほど良くなる単調な傾向はなかった。また、Hごとに追加学習を始めた時点の重みも違い、計算単位の数だけが原因とは断定できない。

H40のCSRは最小容量・最短時間、H100のCSRは最大の容量削減率・時間短縮率となったが、いずれも精度維持を同時に達成していない。使用目的によって許容誤差を定め、今回のMSE悪化を含めて判断する必要がある。

7.5 計画との差分と限界

物理的な10回反復と、全8条件の単独Flash・静的RAMビルド比較は実施した。一方、未見チャートの独立最終評価、当初計画のH50、各方式の搭載限界、実行時スタックの最大量は未測定である。紙は手持ちであり、距離・角度・照度を数値で統制していない。同じ紙と装置で続けて読んだ10回であり、様々な場所や日付で使う場合を代表する測定とはいえない。

学習も当初の33組・MSE損失から、135行・重み付きSmooth L1へ変更した。全学習行での通常の重みなしMSEは付録に示し、10回の再読取結果や学習損失と混ぜない。こうした変更と制約を含めて、本結果は実施した条件内の比較として位置づける。

7.6 今後の改善（Future work）

1. 色の正確さを優先して削る量を見直す。 まず閾値を下げ、残す重みを増やしたときに誤差が減るか調べる。次に、層ごとに閾値を選ぶ方法や、削った接続を0に固定して追加学習する方法を試す。重みが小さくても、後の計算で影響が大きくなる場合があるため、出力の誤差も確認する。

これらは新しい実験として行い、学習用のデータで学び、検証用のデータで設定を選ぶ。既に得た10回の結果へ合わせ込まず、設定を固定した後に別の最終チャートでMSE300以下と増加5%以下を判定する。

2. 入力で色の区別が消えていないか調べる。 検証128入力には、正規化後が同じ(0,0,0)となる6行があり、正解の色は6種類だった。同じ入力には同じ計算結果しか出せないため、モデルを大きくするだけでは、この6色を区別できない。

黒白の基準を読み直した差、RAW、クリップ前の入力、0や1へ収めた割合を残し、どこで色の情報が失われたか調べる。照明・距離・角度の変動も記録し、その範囲に対応する入力の変化を学習へ加える。改善したかは別の校正・読取で確かめる。

3. 作業用メモリと速度を詳しく調べる。 現在は100個分の中間計算用配列を2本確保している。各Hに必要な個数だけにすれば、宣言上の容量は $8H$ byteとなり、H40なら800→320 byteへ小さくできる。ただし未実装の案であり、実際のスタック最大量がこの値になると確認したわけではない。

今後は、数値の配置を示すリンカマップと実行時の最大スタック使用量を記録する。そのうえで、位置情報を表す整数の上限も確かめ、何次元まで搭載できるかを調べる。速度は、出力合計や入力選択の時間を分けて測れば、補正計算そのものの短縮をさらに詳しく説明できる。

4. 確かめる条件を増やす。 H50を加えて当初計画の条件を補う。乱数の開始条件や読む日を変え、たまたまよい結果になったのか、繰り返しても同じ傾向になるのかを確認する。H200などへ大きくするときは、使えるメモリと位置情報を表す整数の上限を先に調べる。

最後に、学習に使っていない別の紙を用意し、学習用・調整用・最終確認用の役割を固定して測る。以上は今後試せる方法であり、今回すでに改善を達成した結果ではない。

8　結論

紙の9色を実際に10回読み、通常方式（Dense）と圧縮方式（枝刈り＋CSR）を、H40・60・80・100で比較した。枝刈り＋CSRはモデル配列容量を60.51〜69.70%、固定入力での実機推論時間を28.97〜44.01%削減した。単独ビルドのFlash使用量は7.94〜31.26%減少し、静的RAMは全条件で6,764 byteと変わらなかった。一方、10回平均MSEは全次元で増加し、精度維持およびCSRの平均MSE300以下は達成しなかった。

達成できたのは、モデルを小さくして計算を速くすることだった。達成しなかったのは、圧縮後も色の正確さを保つことだった。 したがって、今回の条件では容量と速度の改善を確認したが、精度維持を含む目的を全面的に達成したとはいえない。今後は入力の再現性を確認し、評価条件を分けて閾値を決め、独立した未見チャートで精度を検証する必要がある。

9　参考文献

[1] R. Tibshirani, “Regression Shrinkage and Selection via the Lasso,” Journal of the Royal Statistical Society: Series B, 58(1), pp. 267–288, 1996.

[2] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning Both Weights and Connections for Efficient Neural Networks,” Advances in Neural Information Processing Systems 28, pp. 1135–1143, 2015.

[3] N. Bell and M. Garland, “Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors,” Proceedings of SC ’09, Article 18, pp. 1–11, 2009.

[4] Arduino, “micros(),” Arduino Documentation, https://docs.arduino.cc/language-reference/en/functions/time/micros/.

[5] PyTorch Contributors, “SmoothL1Loss,” PyTorch Documentation, https://docs.pytorch.org/docs/stable/generated/torch.nn.SmoothL1Loss.html.

10　付録：各回の実測値と再現資料

10.1　全10回のDense MSE

回	H40	H60	H80	H100
1	96.037037	2.666667	12.777778	367.185185
2	2889.148148	2339.370370	2069.259259	2063.629630
3	79.851852	10.481481	89.185185	113.518519
4	17.370370	1.555556	22.888889	2.518519
5	10.296296	11.185185	84.037037	7.222222
6	60.555556	8.037037	24.444444	25.222222
7	12.888889	3.407407	5.296296	5.148148

回	H40	H60	H80	H100
8	5.296296	1.222222	1.259259	13.629630
9	7.259259	3.111111	5.111111	3.740741
10	16.925926	2.333333	2.555556	2.444444

10.2　全10回の枝刈り＋CSR MSE

回	H40	H60	H80	H100
1	228.851852	177.185185	233.814815	846.000000
2	3078.333333	2811.555556	2407.851852	2576.074074
3	217.777778	262.888889	272.629630	513.888889
4	112.703704	228.592593	257.703704	331.370370
5	118.370370	251.222222	328.444444	386.296296
6	182.555556	230.074074	202.444444	474.851852
7	152.555556	250.629630	271.074074	352.703704
8	138.370370	241.037037	206.037037	318.851852
9	125.740741	246.518519	235.037037	377.074074
10	165.148148	235.407407	242.259259	342.185185

10.3　学習135行の重みなしMSE

次の値は学習に使った行をもう一度入力した結果である。「重みなし」は各行に重要度を付けずに平均したという意味で、モデルの重みを消したという意味ではない。新規10回のMSEや学習時のSmooth L1損失とは分けて読む。不安定な過去の入力も保持しているため、大きな値が残っている。

H	学習135行のMSE	うち3×3の111行のMSE
40	1794.73	1049.10
60	1346.71	867.25
80	1151.08	764.50
100	922.77	495.82

10.4　使用ファイルと再現方法

code/teacher_fresh_v10/ に実際の新規読取コードと数値ヘッダ、code/teacher_compare_v10/ に実際の速度比較コードとヘッダを置いた。両者の数値ヘッダは完全一致し、学習データとは別ファイルである。data/ に集計CSV、raw_logs/ に元の受領ログ、source_snapshot/ にV10と必要な補助ソース・親重みを保存した。manifest.json に各ファイルを同じものか照合するための識別値（SHA256）を記録した。

正解は reference/result.ppm で、SHA256は 582b783b099db5bd6fae89ee0355a17f427fb4a4fc3f2750a8acb8398e5383e2 である。学習135行のSHA256は 67385d38c581978bde6bacc2d4571283d0198573939cc8eebc6bea39e6ade73a、数値ヘッダは 2516a64a3acea49c10a2a26e4c8ffa5f8e1f5092270c7fba0ece0b02b4edcb4e である。

新規10回は fresh_readings/capture_01 から capture_10 の元ログに対応する。保存済み入力の実機再生・速度・使用量は hardware/run_01 の別記録であり、新規10回へ追加カウントしていない。

単独モデルのFlash・静的RAMは hardware/single_model_memory_run_01 の利用者受領文と条件別CSVに対応する。使用コードは code/memory_probe_v10/、集計は data/single_model_memory_results.csv と data/single_model_memory_comparison.csv に保存した。

再現用コードはPythonによる学習・枝刈り・CSR出力・ログ検算とC++推論コードを含む。新しい紙の読み取り値は同梱ログから再計算できるが、プログラムを実行するだけで新たな物理測定を作ることはできない。ファイル対応と実行手順は同梱のREADMEに記載した。